FLUTTER COURSE · SESSION 9 OF 15

Firestore Database

Last session, people could log in. Now their data goes to the cloud: it belongs to their account, appears on every device they use, and updates live the moment it changes. Then you'll lock it down so nobody can read anyone else's data.

BY THE END OF TODAY YOU CAN

Create, read, update and delete documents in Firestore, show them live with a Stream, store data per user with a `userId` field, and write security rules so users can only see their own data.

The big picture: a filing cabinet in the cloud

Firestore is Firebase's database. It isn't a table like Excel; it's a **filing cabinet**. Each **drawer** is a **collection**. Inside each drawer are **documents**, each one like a single sheet of paper with labelled boxes (**fields**) on it. Every sheet has a unique **id**.

collection: notes

```
id: a8Fk2Lq…
title: "Revise Riverpod"
body: "watch vs read"
userId: "u7Hs…"
createdAt: 16 Oct, 10:42
```

```
id: Zp91mXc…
title: "Buy milk"
body: ""
userId: "k2Qa…"
createdAt: 16 Oct, 11:05
```

collection: shoppingItems

```
id: 3Jd8sWq…
name: "Rice 5 kg"
done: false
userId: "u7Hs…"
```

Two more ideas make Firestore special:

- **It's live.** When a document changes, every phone and browser that is watching sees the change within a second, like a shared Google Doc.
- **It has a security guard. Security rules** check the ID badge from Session 8 before letting anyone read or write. Without rules, anyone could read everything.

Setup

1. Firebase console → **Build → Firestore Database → Create database**. Pick a location close to your users (e.g. `asia-south1`, Mumbai), then **Start in production mode**. We'll write our own rules today.
2. Add the package: `flutter pub add cloud_firestore`
3. Keep everything from Session 8: `Firebase.initializeApp` in `main()`, and the auth gate. Firestore data belongs to the logged-in user.

Firestore words, in plain English

WORD	WHAT IT MEANS
Collection	A drawer of documents, e.g. `notes`.
Document	One record, e.g. one note. Has an **id** and **fields**.
Field	One named value in a document: `title`, `done`, `createdAt`...
`FirebaseFirestore.instance`	The cabinet. Every call starts here.
`.collection('notes')`	Open the `notes` drawer.
`.doc(id)`	Pick one sheet by its id.
`.snapshots()`	A **Stream** that sends the documents again every time anything changes. This is what makes the app live.
`QuerySnapshot` / `.docs`	One delivery from that stream: the list of documents right now.
`FieldValue.serverTimestamp()`	"Use Firebase's clock for this time", so every user gets the same, correct time.
Security rules	The guard. Decide who may read and write each document, using `request.auth.uid` (the logged-in user's id).

The concepts, in the order we cover them

1 CRUD: create, read, update, delete

Create	Read once	Read live	Update
`notes.add({...})`	`notes.doc(id).get()`	`notes.snapshots()`	`notes.doc(id).update({.`

Adds a document. Firestore makes the

Reads one document a single time.

Keeps sending the latest documents.

Changes only the fields give.

Delete

`notes.doc(id).delete()`

Removes the document.

```dart
final notes = FirebaseFirestore.instance.collection('notes');
final uid = FirebaseAuth.instance.currentUser!.uid;     // who is logged in (Session 8)

// Create
await notes.add({
  'title': title,
  'body': body,
  'userId': uid,                                        // ← whose note this is
  'createdAt': FieldValue.serverTimestamp(),
});

// Update: only the fields you list change
await notes.doc(note.id).update({'title': newTitle, 'body': newBody});

// Delete
await notes.doc(note.id).delete();
```

---

2 A model class for documents

Same idea as Session 6's `fromJson`: read the document **once**, in one place. The document's **id** isn't inside its fields, so we pass it in separately.

```dart
class Note {
  const Note({required this.id, required this.title, required this.body,
this.createdAt});
  final String id;
  final String title;
  final String body;
  final DateTime? createdAt;

  factory Note.fromFirestore(DocumentSnapshot<Map<String, dynamic>> doc) {
    final data = doc.data()!;
    return Note(
      id: doc.id,                                          // the id comes from the
document
      title: data['title'] as String,
      body: data['body'] as String? ?? '',
      createdAt: (data['createdAt'] as Timestamp?)?.toDate(),   // can be null for a
moment, see below
    );
  }
}
```

WHY CREATEDAT CAN BE NULL

When you add a note, Firestore shows it on your screen **straight away**, before the server has stamped its time. For that first moment `createdAt` is `null`. A second later the real time arrives and the stream sends the note again. Always make the time nullable (`Timestamp?`) and show “Just now” when it's null.

---

3 Live updates: StreamBuilder and StreamProvider

`.snapshots()` is a Stream: it sends the whole list again whenever anything changes, on any device. Show it with a `StreamBuilder` (the Flutter way) or a `StreamProvider` (the Riverpod way, like Session 8's auth gate).

```
// Only MY notes, newest first
final myNotesProvider = StreamProvider<List<Note>>((ref) {
  final uid = ref.watch(authStateProvider).value?.uid;     // from Session 8
  if (uid == null) return Stream.value([]);                // logged out → no notes

  return FirebaseFirestore.instance
      .collection('notes')
      .where('userId', isEqualTo: uid)
      .orderBy('createdAt', descending: true)
      .snapshots()
      .map((snapshot) => snapshot.docs.map(Note.fromFirestore).toList());
});

// In a ConsumerWidget, exactly like Session 6:
ref.watch(myNotesProvider).when(
  loading: () => Spinner(),
  error: (e, _) => ErrorView(e),
  data: (notes) => notes.isEmpty ? EmptyState() : NotesList(notes),
);
```

Try this in class: open the app in two windows (or on a phone and in Chrome), logged in as the same user. Add a note in one, and it appears in the other within a second. Open the Firebase console's **Data** tab too: it updates live as well.

---

4 Queries: where, orderBy, limit

CODE	MEANS
`.where('userId', isEqualTo: uid)`	Only documents whose `userId` is mine
`.where('done', isEqualTo: false)`	Only items not done yet
`.orderBy('createdAt', descending: true)`	Newest first
`.limit(20)`	At most 20 documents

THE INDEX ERROR (EVERYONE GETS THIS ONCE)

A query that filters on one field (`userId`) and sorts by another (`createdAt`) needs an **index**. The first time, the app shows an error saying *"The query requires an index"* with a **link**. Open the link, click **Create index**, and wait a minute or two. The query then works.

---

5 Security rules: users only see their own data

The Flutter code filters by `userId`, but that only protects honest apps. Someone could write their own code and ask for **everyone's** notes. Rules run on Firebase's servers, so they can't be skipped.

When you create the database you choose one of two starting points, and **neither is safe for a real app**:

MODE	DEFAULT RULE	PROBLEM
Test mode	`allow read, write: if request.time < …30 days…;`	**Open:** anyone on the internet can read and change everything, for 30 days
Production mode	`allow read, write: if false;`	**Locked:** nobody can read or write anything, including your own app

Write rules that say exactly who may do what. Firebase console → Firestore → **Rules** → paste → **Publish**:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /notes/{noteId} {
      // Read, change or delete a note only if it's yours
      allow read, update, delete: if request.auth != null
                                  && request.auth.uid == resource.data.userId;
      // Create a note only with YOUR id in it
      allow create: if request.auth != null
                    && request.auth.uid == request.resource.data.userId;
    }
  }
}
```

IN THE RULE	MEANS
`request.auth`	The badge of whoever is asking. `null` = not logged in
`request.auth.uid`	Their user id (Session 8's `user.uid`)
`resource.data`	The document **as it is now** in the database
`request.resource.data`	The document **as it would be** after this write (used when creating)

RULES ARE NOT FILTERS

If your query asks for **all** notes, Firebase refuses the whole query with `permission-denied`, even though some of them are yours. The query must ask only for what the rules allow: always include `.where('userId', isEqualTo: uid)`.

Prove it works: register two accounts. Add notes as User A. Log out, log in as User B: B sees none of A's notes. Then try a query without the `where` and watch it fail with `permission-denied`.

Errors you will see today

> `[cloud_firestore/permission-denied]`
>
> Your rules said no. Either the rules are still "locked" (production default), the query doesn't filter by `userId`, or you're not logged in.

> `[cloud_firestore/failed-precondition] The query requires an index`
>
> Open the link in the error message, click Create index, wait 1–2 minutes.

> `"type 'Null' is not a subtype of type 'Timestamp'"`
>
> `createdAt` is null for a moment after adding. Use `as Timestamp?` and handle null.

> `"Null check operator used on a null value" at currentUser!`
>
> Nobody is logged in. Only open data screens from behind the Session 8 auth gate.

> `The list never updates`
>
> You used `.get()` (read once) instead of `.snapshots()` (live).

Practice task (in class): Notes app

- [ ] Start from your Session 8 login app.
- [ ] Add a note (title and body), list your notes newest first, edit a note, and delete a note.
- [ ] Every note stores the logged-in user's `userId`, and the list only shows yours.
- [ ] The list updates live: test in two windows.
- [ ] Publish security rules, then prove with two accounts that User A can't see User B's notes.

Project: Shopping List App

- [ ] Log in (reuse Session 8), then a shopping list for **your** account only.
- [ ] Add an item with a validated text field. Each item stores `name`, `done`, `userId` and `createdAt`.
- [ ] Tick an item to mark it done (an **update**), and swipe or tap to delete it.
- [ ] The list is shown live with a `StreamBuilder` or `StreamProvider`, with loading, empty and error states.
- [ ] Security rules so each user only reads and writes their own items.
- [ ] Bonus: a “Clear done items” button, and a count of items left in the AppBar.

Check yourself

<details>
<summary>What's the difference between a collection and a document?</summary>

A collection is a drawer that holds many documents. A document is one record with an id and fields.
</details>

<details>
<summary>What's the difference between .get() and .snapshots()?</summary>

`.get()` reads once. `.snapshots()` is a Stream that sends the data again every time it changes.
</details>

<details>
<summary>Why do we store userId in every document?</summary>

So we can ask only for the current user's documents, and so the security rules can check that a document belongs to whoever is asking.
</details>

<details>
<summary>Why isn't filtering by userId in Flutter enough to keep data private?</summary>

Anyone can write their own code that skips the filter. Security rules run on Firebase's servers, so they can't be skipped.
</details>

<details>
<summary>Why does a query for all notes fail even if some are mine?</summary>

Rules aren't filters. The query must ask only for documents the rules allow, so it has to include `where('userId', isEqualTo: uid)`.
</details>

NEXT SESSION

Session 10 · Polishing UI/UX. Light and dark themes with a saved toggle, Hero transitions, animations, responsive layouts, and your own app icon and splash screen.